

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ PÓS-GRADUAÇÃO ENGENHARIA DE SOFTWARE, DEVOPS E TRANSFORMAÇÃO DIGITAL

DISCIPLINA: Desafio na Trilha de Gestão e Inovação(Turma U)

Aluna: Josiane Fatima de Souza

Atividade 02 - Individual

ETAPA 2

CANVAS DE ARQUITETURA

Visão Arquitetural de Alto Nível- Sistema de RH do TER/PR

1. Objetivo do Sistema

Modernizar o ecossistema de Gestão de Pessoas do TRE/PR, eliminando a fragmentação entre sistemas legados locais e plataformas nacionais do TSE, entregando uma experiência digital unificada de autoatendimento ao servidor com automação ponta a ponta de processos (férias, licenças, averbações) e integração consistente com eSocial, SGP Nacional e SEI.

Resultado: reduzir retrabalho, encurtar o ciclo de concessão de benefícios e garantir conformidade contínua com TCU, CNJ e TSE.

2. Stakeholders

- **Servidores do TRE/PR:** usuários do autoatendimento
- **Secretaria de Gestão de Pessoas (SGP):** donos do negócio
- **Secretaria de TI (STI):** executores e operação
- **TSE:** autoridade central / SGP Nacional
- **Órgãos de controle:** TCU, CNJ, Receita Federal (eSocial)
- **Alta administração do Tribunal:** patrocínio

3. Restrições

- Dependência de cronograma e esquemas do TSE
- Conformidade obrigatória com eSocial e LGPD
- Folha de pagamento é serviço crítico — zero tolerância a indisponibilidade

- Aderência à IN nº 1/2019 e normas do CNJ
- Janelas estritas de manutenção e homologação
- Stack corporativa preferencial: Java/Spring + Oracle/PostgreSQL

4. **Requisitos Funcionais Essenciais**

- Cadastro e ciclo de vida do servidor (admissão, movimentação, desligamento)
- Folha de pagamento com vantagens locais e decisões judiciais específicas
- Autoatendimento: férias, licenças, averbações e declarações
- Workflow de aprovação eletrônica integrado/substitutivo ao SEI
- Gestão de frequência, banco de horas e teletrabalho
- Geração de eventos para eSocial e sincronização com SGP Nacional do TSE
- Relatórios gerenciais, indicadores de força de trabalho e trilhas de auditoria

5. **Requisitos de Qualidade (ISO/IEC 25010)**

- **Confiabilidade:** disponibilidade $\geq 99,5\%$ na folha
- **Segurança:** AuthN/AuthZ corporativa, LGPD, cifragem em trânsito e repouso
- **Manutenibilidade:** modularidade para absorver mudanças jurídicas frequentes
- **Compatibilidade:** interoperabilidade com TSE, eSocial, SEI
- **Performance:** processamento da folha dentro da janela mensal
- **Usabilidade:** portal mobile-friendly e acessível (eMAG/WCAG 2.1 AA)
- **Portabilidade:** containerização para nuvem privada do Judiciário

6. **Integrações Externas**

- TSE — SGP Nacional (REST)
- eSocial — Receita Federal (SOAP/REST)
- SEI — processos administrativos
- Sistema de Identidade Corporativa (SSO/LDAP)
- Power BI / Data Lake do Tribunal
- Sistemas bancários para crédito da folha
- PJe e demais sistemas do CNJ (consulta)

7. Decisões Arquiteturais Iniciais

- **Strangler Fig:** decompor o monólito Java/Oracle de forma incremental, sem big-bang
- **Microserviços por domínio:** Folha, Frequência, Benefícios, Autoatendimento
- **API Gateway + BFF:** expor capacidades ao Portal do Servidor (web/mobile)
- **Arquitetura orientada a eventos:** Kafka/RabbitMQ para sincronização quase em tempo real
- **Replace antes de Rebuild:** substituir módulos locais pelo SGP Nacional quando homologado
- **Regras jurídicas isoladas:** engine de regras desacoplado da folha (Drools ou similar)
- **Observabilidade nativa:** OpenTelemetry, logs estruturados, métricas SRE
- **CI/CD com feature flags:** deploys seguros mesmo em serviço crítico

8. Riscos Arquiteturais

- **Dependência do TSE:** mudanças de esquema podem invalidar contratos de integração
- **Quebra da folha:** alterações no monólito têm efeito colateral imprevisto
- **Regras jurídicas presas em stored procedures:** alto custo de migração
- **Sincronização local ↔ nacional:** risco de divergência de dados mestres do servidor
- **Sombra do SEI:** processos paralelos podem persistir e gerar retrabalho
- **Capacidade do time:** curva de aprendizado em microserviços e eventos
- **LGPD:** exposição indevida de dados sensíveis em integrações
- **Lock-in de fornecedor:** soluções proprietárias de nuvem fora do Judiciário

9. Esboço da Stack Tecnológica

Backend

Java 21 · Spring Boot 3 · Spring Cloud

Frontend

React + TypeScript · Design System próprio

Dados

PostgreSQL · Oracle (legado) · Redis (cache)

Mensageria

Apache Kafka · Outbox Pattern

Plataforma

Kubernetes · Docker · Helm · GitLab CI/CD

Observabilidade

OpenTelemetry · Grafana · Loki · Prometheus

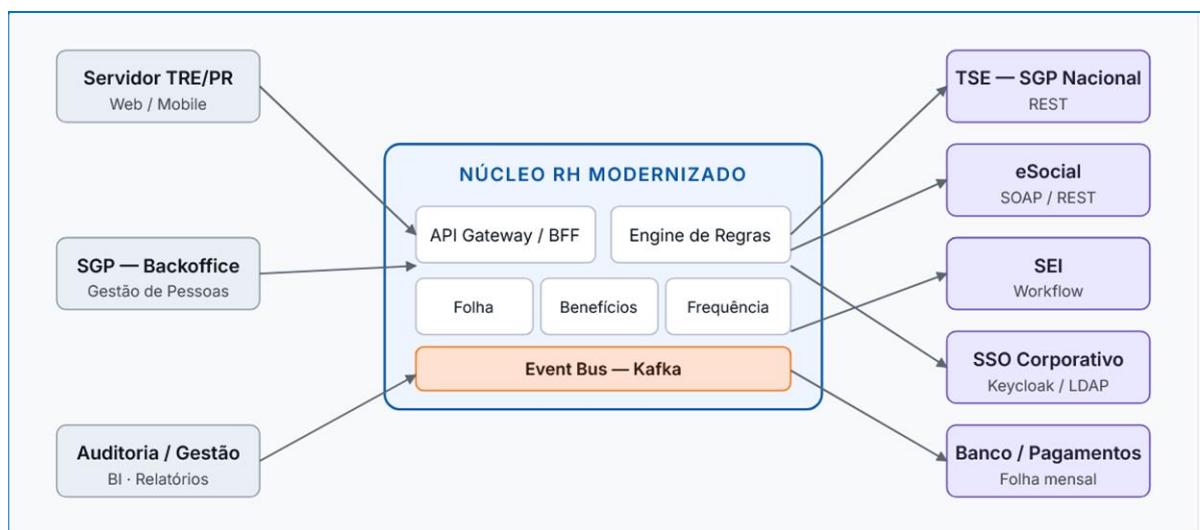
Segurança

Keycloak (SSO/OIDC) · Vault · LGPD-by-design

Integrações

API Gateway (Kong) · Camel · REST/SOAP

10. Visão de Contexto Preliminar



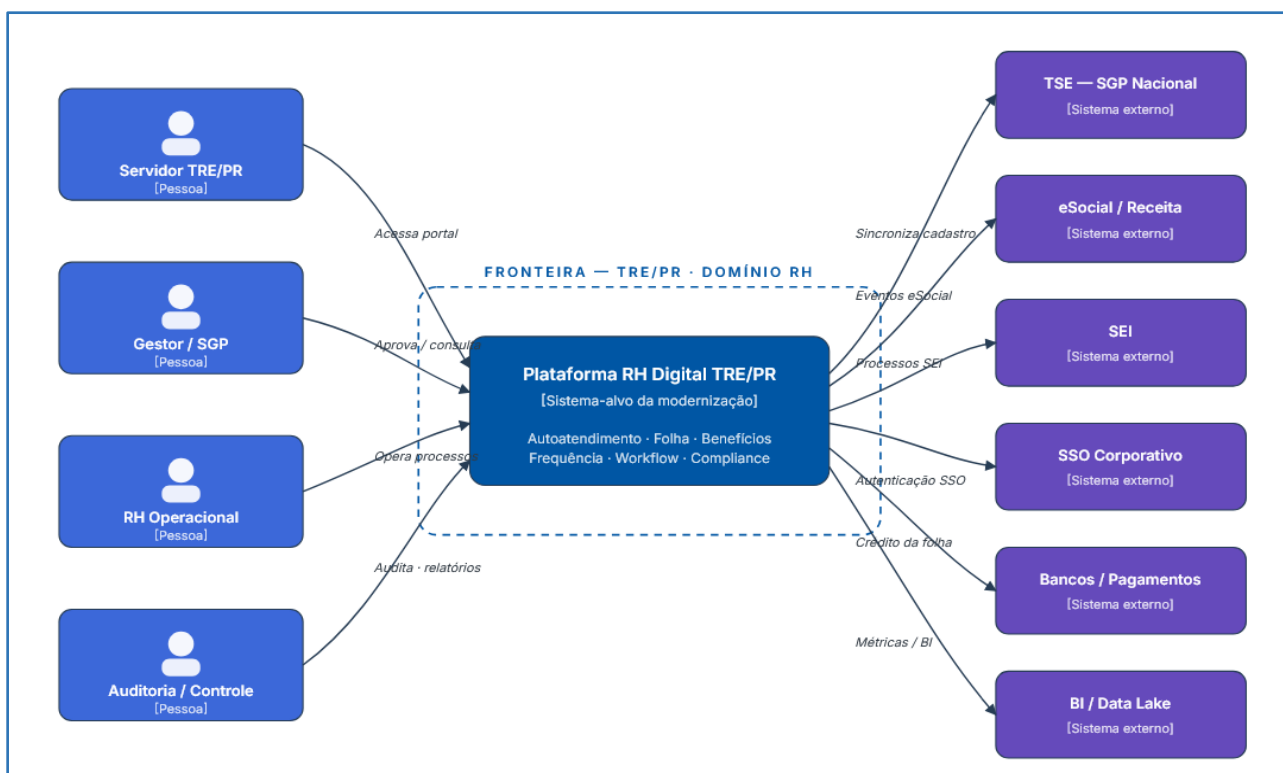
O **Portal do Servidor** e os **backoffices da SGP** consomem o novo núcleo de RH via **API Gateway**. O núcleo orquestra microsserviços de domínio, publica eventos no barramento e mantém integrações controladas com sistemas externos do TSE, Receita Federal e SEI.

C4 — Nível 1

Diagrama de Contexto do Sistema

Usuários, sistemas externos, fronteira do domínio e fluxos de interação macro.

- Pessoa— usuário do sistema
- Sistema-alvo— modernizado
- Sistema externo



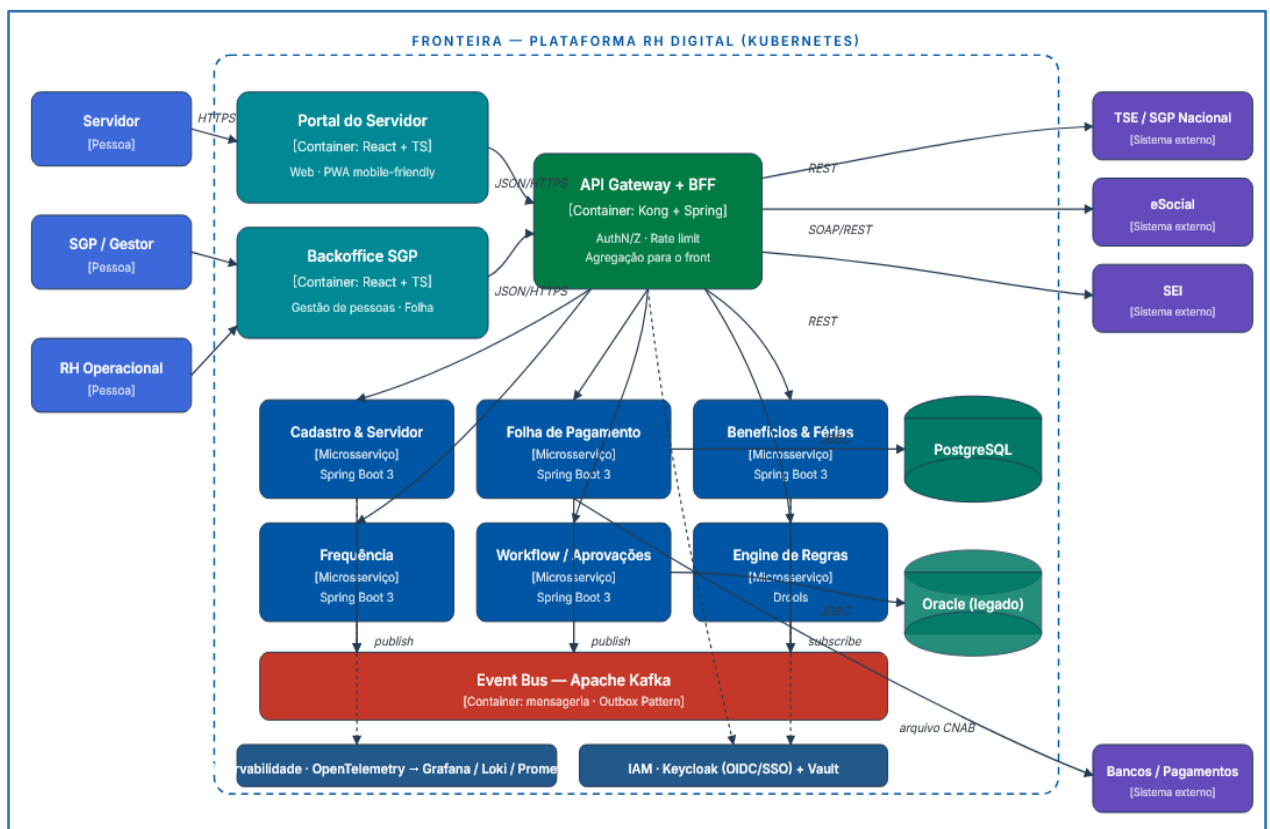
Fronteira: O sistema-alvo opera dentro da fronteira lógica do TRE/PR — Domínio RH, consolidando capacidades hoje fragmentadas entre módulos locais e plataformas nacionais.

Fluxos macro: Servidores e gestores interagem via portal; o núcleo se integra ao TSE (cadastro nacional), eSocial (eventos trabalhistas), SEI (processos), SSO corporativo (identidade) e bancos (folha mensal), além de alimentar o BI institucional.

Diagrama de Containers

Frontend, API/BFF, microsserviços de domínio, bancos, mensageria e observabilidade.

- Frontend
- API / BFF
- Microsserviço
- Banco de dados
- Event bus
- Observabilidade / IAM
- Sistema externo



Frontend: Portal do Servidor (PWA React/TS) para autoatendimento e Backoffice SGP para gestão de pessoas e folha. Ambos consomem a API/BFF via HTTPS/JSON.

API / BFF: Kong como API Gateway (AuthN/Z, rate limit, roteamento) e um BFF Spring que agrega chamadas aos microsserviços para reduzir chattiness no front.

Camada de serviços: Microsserviços por subdomínio - Cadastro, Folha, Benefícios, Frequência, Workflow e a Engine de Regras (Drools) isolando a lógica jurídica e fiscal.

Bancos de dados: PostgreSQL por serviço (database-per-service) e Oracle legado mantido temporariamente sob padrão Strangler Fig durante a migração incremental.

Event Bus: Apache Kafka com Outbox Pattern para propagar eventos de domínio (ex.: *FolhaProcessada*, *FeriasAprovadas*) e desacoplar integrações com TSE e eSocial.

Observabilidade & IAM: OpenTelemetry → Prometheus (métricas), Loki (logs) e Grafana (dashboards/alertas). Keycloak centraliza OIDC/SSO e Vault guarda segredos.

Introdução e Objetivos

O Sistema de Gestão de Pessoas do TRE/PR opera hoje como um conjunto fragmentado de módulos legados em Java/Oracle, integrado de forma frágil a plataformas nacionais do TSE (SGP), ao eSocial e ao SEI. A modernização proposta tem como objetivo entregar uma plataforma digital unificada de RH, com autoatendimento ao servidor, automação ponta a ponta de processos (férias, licenças, averbações), folha de pagamento resiliente e conformidade contínua com TCU, CNJ, TSE e LGPD.

Metas-chave: reduzir retrabalho operacional da SGP, encurtar o ciclo de concessão de benefícios, eliminar divergências entre base local e SGP Nacional, e habilitar evolução incremental sem big-bang.

Drivers e Requisitos Arquiteturais

Drivers de negócio

- Pressão por digitalização do serviço público
- Adesão obrigatória ao eSocial e ao SGP Nacional
- Redução de custo operacional na SGP
- Experiência do servidor (mobile, autoatendimento)

Requisitos funcionais essenciais

- Ciclo de vida do servidor (admissão a desligamento)
- Folha de pagamento com regras locais e judiciais
- Autoatendimento: férias, licenças, averbações
- Workflow eletrônico integrado/substitutivo ao SEI
- Frequência, banco de horas e teletrabalho
- Sincronização com SGP Nacional e eventos eSocial

QUALIDADES CRÍTICAS (ISO/IEC 25010)

- Confiabilidade: disponibilidade $\geq 99,5\%$ para folha; janela mensal inviolável
- Segurança: SSO corporativo, LGPD-by-design, cifragem em trânsito e repouso
- Manutenibilidade: regras jurídicas isoladas em engine, fora da folha
- Compatibilidade: interoperabilidade com TSE, eSocial, SEI, bancos
- Performance: processamento da folha dentro da janela operacional mensal
- Usabilidade: portal mobile-first e acessível (eMAG / WCAG 2.1 AA)

VISÃO DE SOLUÇÃO (LIGADA AO C4 NÍVEL 2)

A solução adota uma arquitetura de **microsserviços por domínio** orquestrados por um **API Gateway + BFF**, com **barramento de eventos Kafka** para integração assíncrona e padrão **Strangler Fig** para substituir incrementalmente o monólito Oracle.

FRONTEND: Portal do Servidor (React) + Backoffice SGP

API / BFF: Kong Gateway + BFF Spring Boot

SERVIÇOS DE DOMÍNIO: Cadastro · Folha · Benefícios · Frequência · Workflow · Regras (Drools)

DADOS: PostgreSQL (novo) + Oracle (legado, em substituição)

MENSAGERIA: Apache Kafka + Outbox Pattern

OBSERVABILIDADE & IAM: OpenTelemetry · Grafana/Loki · Keycloak · Vault

Decisões Arquiteturais (ADRs simplificados)

ADR-001 · Strangler Fig sobre o monólito

Contexto: Monólito Java/Oracle concentra folha e cadastros; substituição big-bang é proibitiva e arriscada.

Decisão: Decompor por domínio incrementalmente, mantendo o legado em produção até cada serviço novo absorver suas responsabilidades.

Consequência: Convivência temporária de dois bancos; necessidade de sincronização e contratos versionados.

ADR-002 · Microserviços por domínio de RH

Contexto: Domínios (Folha, Benefícios, Frequência) têm ciclos de mudança e criticidade distintos.

Decisão: Serviços independentes por bounded context, deploy isolado, banco próprio por serviço.

Consequência: Maior complexidade operacional; exige plataforma Kubernetes e observabilidade madura.

ADR-003 · Event bus Kafka + Outbox

Contexto: Integrações síncronas entre módulos causam acoplamento e falhas em cascata.

Decisão: Comunicação assíncrona via Kafka para eventos de domínio; Outbox para consistência transacional.

Consequência: Curva de aprendizado em mensageria; necessidade de schema registry e governança de eventos.

ADR-004 · Engine de regras desacoplado (Drools)

Contexto: Regras jurídicas mudam por decisão judicial e estão hoje em stored procedures Oracle.

Decisão: Externalizar regras em engine dedicado, versionado e testável fora da folha.

Consequência: Auditabilidade e velocidade de mudança; exige equipe treinada em DRL e governança de regras.

ADR-005 · SSO corporativo via Keycloak (OIDC)

Contexto: Múltiplos sistemas exigem identidade unificada e MFA conforme LGPD/CNJ.

Decisão: Keycloak como IdP central, federado ao LDAP do Tribunal.

Consequência: Ponto único de autenticação a ser endurecido; simplifica auditoria e acessos.

ADR-006 · API Gateway + BFF dedicado

Contexto: Frontends (Portal e Backoffice) têm necessidades distintas de agregação e segurança.

Decisão: Kong como gateway de borda + BFF Spring por frontend para compor chamadas e aplicar políticas.

Consequência: Camada extra a manter; reduz acoplamento entre UI e serviços de domínio.

Riscos Técnicos

- **Quebra da folha:** qualquer falha tem impacto financeiro e reputacional direto
- **Divergência local ↔ SGP Nacional:** duas fontes de verdade durante a transição
- **Mudança de esquema do TSE:** contratos externos fora do nosso controle podem quebrar integrações
- **Regras presas em stored procedures:** alto custo e risco na migração para o engine
- **Maturidade do time em microserviços/Kafka:** curva de aprendizado pode atrasar entregas iniciais
- **LGPD em integrações:** exposição indevida de dados sensíveis entre sistemas
- **Sombra do SEI:** processos paralelos podem persistir e gerar retrabalho

VISÃO FUTURA

- **Curto prazo (0–6 meses):** Portal do Servidor + autoatendimento de férias/licenças; SSO unificado
- **Médio prazo (6–18 meses):** extração dos domínios Cadastro, Frequência e Benefícios do monólito; engine de regras em produção
- **Longo prazo (18–36 meses):** folha modernizada em microserviço com Drools; adoção plena do SGP Nacional onde homologado; descomissionamento do Oracle legado
- **Evoluções:** data lake corporativo para BI de força de trabalho; assistentes de IA para suporte ao servidor e triagem de processos

Etapa 5 — MVA (Minimum Viable Architecture)

Elementos essenciais da MVA

Frontend mínimo

- **App:** SPA React + Vite (TanStack Start) — apenas Portal do Servidor
- **Escopo:** Login, consulta de contracheque, dados cadastrais, solicitação de férias
- **UI:** Tailwind + componentes shadcn — sem design system completo
- **Deploy:** Estático na borda (Cloudflare Pages / Vercel Free)

API / Backend único

- **Estilo:** Monolito modular REST (sem microsserviços, sem BFF, sem Kafka)
- **Stack:** Node.js + TypeScript (NestJS) OU Java + Spring Boot
- **Camadas:** Controllers → Services → Repositórios (módulos: cadastro, folha, férias)
- **Runtime:** Container único em Cloud Run / Fly.io / Render

Banco único

- **SGBD:** PostgreSQL gerenciado (Neon / Supabase / RDS db.t4g.micro)
- **Modelo:** Schema único com módulos lógicos; migrações via Flyway/Prisma
- **Integração legado:** Carga inicial via ETL pontual a partir do Oracle do SGP
- **Backup:** Snapshots diários automáticos do provedor gerenciado

Autenticação simples

- **Provedor:** Keycloak embarcado OU Supabase Auth (e-mail + senha)
- **Protocolo:** OIDC com JWT de curta duração + refresh token
- **RBAC:** Dois papéis no MVP: SERVIDOR e RH
- **Federação:** SSO institucional (LDAP/SAML) fica fora do MVP

Logs básicos

- **Formato:** Structured logging JSON (pino / Logback) via stdout
- **Coleta:** Logs nativos do provedor (Cloud Run Logs / Logflare)
- **Retenção:** 7 a 14 dias no plano gratuito; export sob demanda
- **Auditoria:** Tabela audit_log no Postgres para ações sensíveis

Métricas mínimas

- **Health:** Endpoints /health e /ready expostos pela API
- **RED:** Requests, Errors, Duration via middleware Prometheus
- **Dashboard:** Grafana Cloud Free OU métricas nativas do provedor

- **Alertas:** 1 alerta: erro 5xx > 2% por 5min → e-mail/Slack

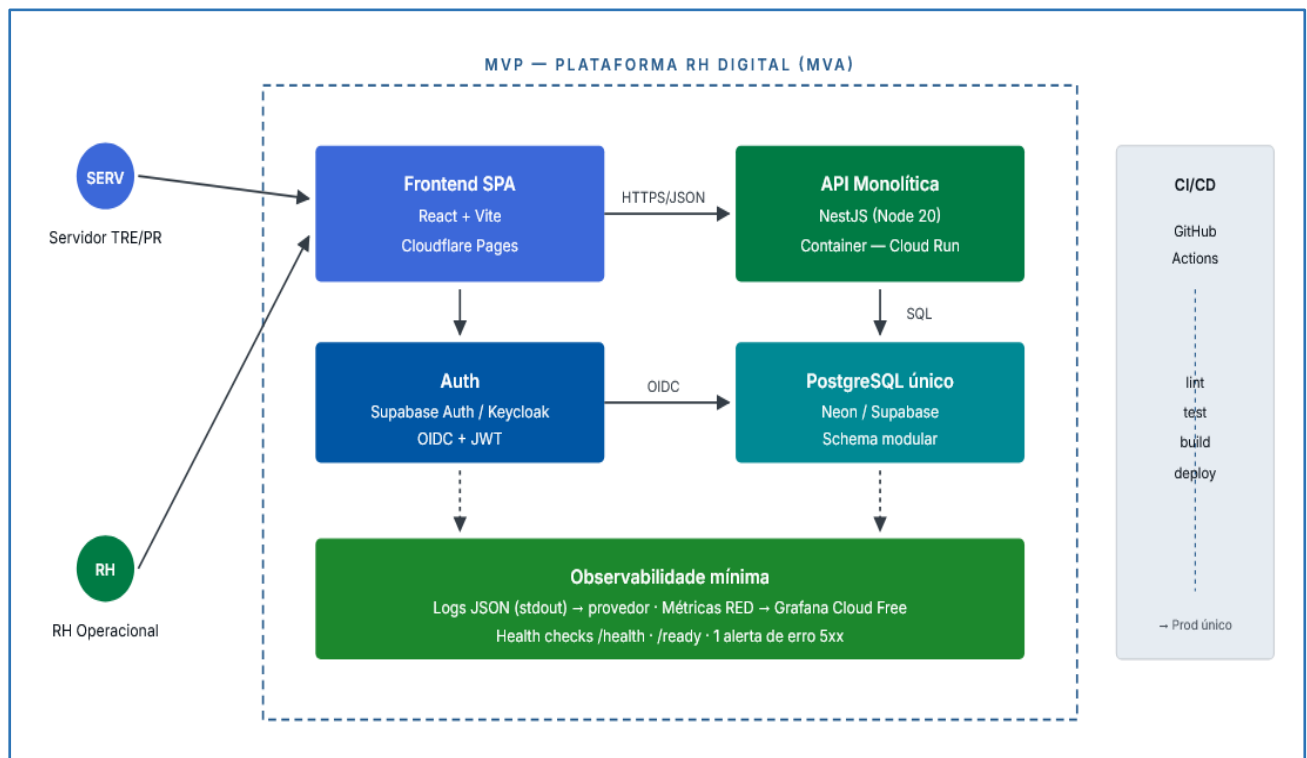
Pipeline CI/CD mínimo

- **VCS:** GitHub (repo único monorepo front + back)
- **CI:** GitHub Actions: lint → test → build → docker push
- **CD:** Deploy automático em push na main para ambiente único (prod-MVP)
- **Qualidade:** Testes unitários essenciais + scan de dependências (npm audit)

Stack tecnológica mínima

- **Frontend:** React 19 + Vite + Tailwind + TanStack Router
- **Backend:** NestJS (Node 20) — container Docker único
- **Banco:** PostgreSQL gerenciado (Neon Free / Supabase Free)
- **Hospedagem:** Cloud Run / Fly.io free tier — escala a zero
- **Observabilidade:** Logs do provedor + Grafana Cloud Free
- **CI/CD:** GitHub Actions (minutos gratuitos)

C4 simplificado — somente elementos MVA



Riscos associados às decisões

Monolito único: Acoplamento entre módulos pode dificultar a evolução para microsserviços (mitigação: separar por módulos/pacotes desde já).

Banco único compartilhado: Ponto único de falha e gargalo de performance (mitigação: backups + réplica de leitura quando necessário).

Auth simplificada: Sem federação com SSO institucional — usuários precisarão de credencial à parte no MVP.

Observabilidade reduzida: Logs com retenção curta e poucas métricas dificultam diagnóstico em produção (mitigação: export periódico).

Deploy direto em produção: Ambiente único (sem staging) aumenta risco de regressão (mitigação: feature flags + testes automatizados).

Dependência de free tier: Limites de cota podem ser atingidos com crescimento de uso (mitigação: monitorar consumo e ter plano de upgrade).

Carga inicial do legado: ETL pontual a partir do Oracle pode trazer inconsistências (mitigação: scripts idempotentes + reconciliação).